

Assignment 1. Foundations

Prof. Hwanjo Yu
CSED342 - Artificial Intelligence

Contact: Seokho Youn (kai0920@postech.ac.kr)

Deadline: September 20, 2026, at 23:59 (50% penalty per day of delay)

General Instructions

Assignment 1 consists of five problems, where Problems 1-4 are written, and Problem 5 is a programming task, with each problem's score indicated alongside it. For the written problems, please submit your answers and solutions in PDF format. You may create the PDF by converting from Overleaf or by taking clear, legible photos of your handwritten solutions and combining them into a single PDF; any issues arising from illegible handwriting are solely your responsibility.

For the programming problem, please note that the assignment has been developed in Python 3.12, so please use **Python 3.12** to implement your code. We recommend using **Conda environment**.¹ You must submit the programming file as `submission.py` without changing the file name.

Ultimately, your final submission must be a `{YOUR_STUDENT_ID}.zip` file that contains both `{YOUR_STUDENT_ID}.pdf` (your written solution) and `submission.py`.

Note: Submissions that do not follow the correct format will incur a **30%** deduction.

When working on your code, please make your modifications only within the designated section in `submission.py`—that is, between the markers

```
# BEGIN_YOUR_CODE
```

and

```
# END_YOUR_CODE
```

You can add other helper functions outside the answer block if you want, but do not import other libraries and do not make changes to files other than `submission.py`. You can solve all the problems without any libraries other than *collections* and *math*.

¹<https://docs.conda.io/projects/conda/en/stable/user-guide/install/index.html>

Your code will be evaluated on two types of test cases, **basic** and **hidden**, which you can see in `grader.py`. Basic tests, which are fully provided, do not stress your code with large inputs or tricky corner cases. Hidden tests are more complex and do stress your code. The inputs of hidden tests are provided in `grader.py`, but the correct outputs are not. To run all the tests, type

```
python grader.py
```

This will tell you only whether you passed the basic tests. The script will alert you if your code takes too long or crashes on the hidden tests, but does not say whether you got the correct output. You can also run a single test (e.g., `5a-0-basic`) by typing

```
python grader.py 5a-0-basic
```

We strongly encourage you to read and understand the test cases, create your own test cases, and not just blindly run `grader.py`.

Problem 1 [10 points]

You repeatedly roll a fair six-sided die until you get a 1 (in which case you stop).

- Each time you roll a 3, you *gain* a points.
- Each time you roll a 6, you *lose* b points.
- Rolls of 2, 4, or 5 neither add nor subtract points.

What is the expected number of points you have at the time you stop? (Express your answer in terms of a and b , and show your work by setting up a recurrence equation.)

Problem 2 [10 points]

We flip a coin 8 times with an unknown bias p (the probability of landing heads). Suppose we observe the sequence $\{H, T, H, H, T, H, H, H\}$, which contains 6 heads and 2 tails. The likelihood of observing this particular sequence is

$$L(p) = p^6 (1 - p)^2.$$

Find the value of $p \in (0, 1)$ that maximizes $L(p)$, prove that this is indeed a maximum, and give an intuitive explanation for the result.

Hint: Instead of differentiating the likelihood directly, work with the log-likelihood function

$$\ell(p) = \ln L(p) = 6 \ln(p) + 2 \ln(1 - p).$$

Problem 3 [10 points]

Let's practice taking gradients, which is a key operation for being able to optimize continuous functions. For $\mathbf{w} \in \mathbb{R}^d$ and constants $\mathbf{a}_i, \mathbf{b}_j \in \mathbb{R}^d$ and $\lambda \in \mathbb{R}$, define the scalar-valued function

$$f(\mathbf{w}) = \sum_{i=1}^m \sum_{j=1}^n (\mathbf{a}_i^\top \mathbf{w} - \mathbf{b}_j^\top \mathbf{w})^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2,$$

where $\mathbf{w}$, $\mathbf{a}_i$ and $\mathbf{b}_j$ are column vectors (e.g. $\mathbf{w} = (w_1, \dots, w_d)^\top$) and $\|\mathbf{w}\|_2 = \sqrt{\sum_{r=1}^d w_r^2}$ is known as the L_2 norm. Compute the gradient $\nabla_{\mathbf{w}} f(\mathbf{w})$.

Note: the gradient is a d -dimensional vector of the partial derivatives with respect to each w_i :

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial f(\mathbf{w})}{\partial w_d} \right)^\top.$$

If you're not comfortable with vector calculus, first warm up by working out this problem using scalars in place of vectors and derivatives in place of gradients. Not everything for scalars goes through for vectors, but the two should at least be consistent with each other (when $d = 1$). Do not write out summation over dimensions, because that gets tedious.

Problem 4 [10 points]

Suppose we have an $n \times n$ grid of points, with coordinates $\{0, \dots, n-1\} \times \{0, \dots, n-1\}$. We want to place k arbitrary axis-aligned squares on the grid (i.e., the sides of each square are parallel to the axes and have equal length). Each corner of each square must lie on a grid point, but otherwise there are no constraints on the location or size of the squares. Squares are allowed to overlap, coincide, or have zero size (i.e., all four corners may be the same point). Assume that k is a fixed positive integer. How many possible ways are there to place k squares on the grid? Give your answer in asymptotic form as a function of n using *Big-O* notation $O(\cdot)$.

Note: For a fixed positive integer k , it is unnecessary to consider whether order matters in this problem, since either convention results in the same asymptotic dependence on n .

Problem 5 - Programming [24 points]

If you're new to Python, the following articles provide pointers to various tutorials and examples for the language:

- Python for Programmers:
<https://wiki.python.org/moin/BeginnersGuide/Programmers>
- Example programs of increasing complexity:
<https://wiki.python.org/moin/SimplePrograms>

Problem 5a [4 points]

Implement *find_longest_lexicographically_first_word* in `submission.py`.

Problem 5b [3 points]

Implement *manhattan_distance* in `submission.py`.

Problem 5c [6 points]

Implement *mutate_sentences* in `submission.py`.

Problem 5d [3 points]

Implement *sparse_vector_dot_product* in `submission.py`.

Problem 5e [3 points]

Implement *increment_sparse_vector* in `submission.py`.

Problem 5f [5 points]

Implement *find_most_frequent_words* in `submission.py`.